hevm, new frontiers in symbolic execution of EVM bytecode

dxo, *Mate Soos*, Martin Blicha, Zoe Paraskevopoulou,
with help from the Echidna team, esp. Gustavo Grieco

Argot Collective (`https://argot.org`)

19th of November 2025, Devconnect Argentina

Outline

1. Symbolic Execution
2. hevm's Symbolic Executor
3. Improvements Since Devcon 2024

Symbolic Execution: straight line program

Most execution works by running instructions concretely:

```
---              ax: 1    , bx: 2
mov %bx %ax      ax: 2    , bx: 2
add %ax $4       ax: 6    , bx: 2
```

Symbolic execution, with symbolic state:

```
---              ax: v1   , bx: v2
mov %bx %ax      ax: v2   , bx: v2
add %ax $4       ax: v2+4, bx: v2
```

Symbolic Execution – branching

Concrete execution:

```
-----------        ax: 1     bx: 1
cmp %ax %bx        ax: 1     bx: 1
je .if_true
; false
add %ax $4
jmp short .end
.if_true:
add %ax $5         ax: 6     bx: 1
.end:
```

Symbolic execution, with symbolic state:

```
-----------        ax: v1    bx: v2
cmp %ax %bx
je .if_true
; false
add %ax $4         ax: v1+4  bx: v2
jmp short .end
.if_true:
add %ax $5         ax: v1+5  bx: v2
.end:
-**- v1==v2 ->     ax: v1+5  bx: v2
-**- v1!=v2 ->     ax: v1+4  bx: v2
```

For symbolic execution, we end up having to follow two executions. This can become exponential.

Related Work

Symbolic execution is used in two major ways. One is to **validate static code analysis** results, the other is **pure symbolic execution**. The first approach is followed by Oyente, sCompile, Mythril, etc. These are typically incomplete, and false positives are allowed.

Purely symbolic execution-based systems:

- **Certora Prover**: Based on backwards exploration and weakest precondition computation, with its own specification language
- **KEVM**: K-framework based, allows to "break out" into K to prove lemmas
- **halmos**: Most similar to hevm, written in python, using z3 as its IR
- **EthBMC**: Bounded model checking-based exploration of contracts

hevm's Symbolic Executor

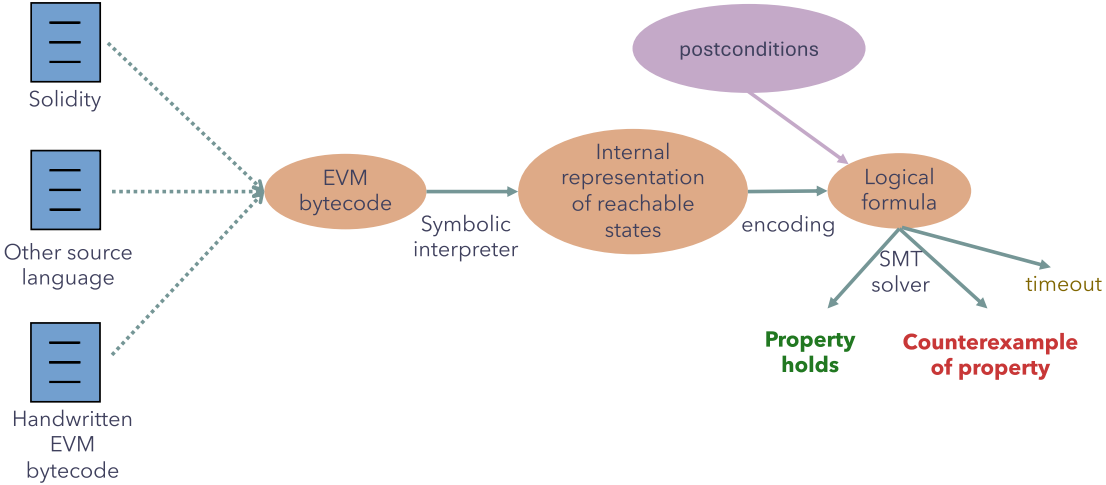
hevm's symbolic executor is very powerful:

- **Operates on bytecode** so runs everything deployed to the chain
- Implements EVM semantics for concrete and symbolic execution
- Understands **all of EVM**: stack, call frames, memory, storage, calldata
- Can run on **any point in blockchain history** via RPC
- Runs an **SMT solver** to find input to reach counterexamples

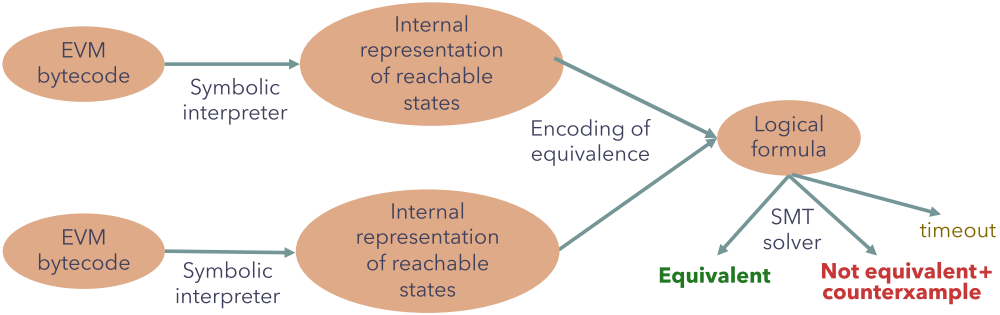
Limitations:

- Cannot deal with **symbolic gas** or **symbolic code** other than by overapproximation
- **Symbolic offset/size memcopy** is not implemented, but is often unneeded
- **Loops and recursion** are explored only to a fixed depth

hevm: Symbolic Execution for Counterexample Generation



hevm: Symbolic Execution for Equivalence Checking



Example: The Great Verification of ABDK Math 64.64

An effort to verify mathematical properties in ABDK Math 64x64 using echidna/hevm.[1]

Invariant	Result
`prove_abs_negative(int128)`	☑
`prove_abs_positive(int128)`	☑
`prove_abs_subadditivity(int128,int128)`	☑
`prove_add_associative(int128,int128,int128)`	☑
`prove_add_commutative(int128,int128)`	☑
`prove_add_identity(int128)`	☑
`prove_add_range(int128,int128)`	☑
`prove_add_values(int128,int128)`	☑
`prove_avg_one_value(int128)`	☑

Invariant	Result
`prove_mul_associative(int128, int128, int128)`	👍
`prove_mul_values(int128, int128)`	👍
`prove_abs_multiplicativeness(int128, int128)`	👍
`prove_inv_division_noncommutativity(int128, int128)`	👍
`prove_div_negative_divisor(int128,int128)`	👍
`prove_gavg_one_value(int128)`	👍
`prove_div_values(int128,int128)`	👍
`prove_inv_double_inverse(int128)`	👍

Verified: 34/57 (**59.65**%)

`https://github.com/gustavo-grieco/abdk-math-64.64-verification`

[1] ABDK Math 64x64: Library of mathematical functions operating with signed 64.64-bit fixed point numbers.

$(x + y) + z = x + (y + z)$

```
function prove_add_associative(int128 x, int128 y, int128 z) public pure {
    int128 x_y = add(x, y);
    int128 y_z = add(y, z);
    int128 xy_z = add(x_y, z);
    int128 x_yz = add(x, y_z);

    assert(xy_z == x_yz);
}
```

```
[RUNNING] prove_add_associative(int128,int128,int128)
Exploring call prefix 0x6727ee5c
Flattening expression
Exploration finished, 14 branch(es) to check in call prefix 0x6727ee5c
Keccak preimages in state: 0
Checking for reachability of 1 potential property violation(s) in call prefix 0x6727ee5c
SMT result: Qed
```

Unique Selling Points of hevm

It is **easy to use**:

- Uses forge-std cheatcodes for easy test setup
- Works on any EVM contract deployed on any chain supported by an RPC node
- No need to recompile contracts, or have their source code

Can be used as a **library**:

- Echidna uses hevm's concrete execution for its fuzzer
- New version of Echidna will be able to also use hevm's symbolic execution engine to complement its fuzzer
- Such combined approach allows it to combine the "best of both worlds"

Can be used as an **equivalence checker**:

- Used by Act, a declarative smart contract language, to check equivalence of the declarative description against the actual compiled contract
- Used by Solidity's fuzzer infrastructure to check compiler optimizations

Equivalence Checking with hevm

hevm compares all externally visible behaviours of two contracts:

- state
- return values
- reverts

but not events, gas usage, or intermediate states and values.

Original function

```
function f(uint x) returns (uint) {
    for(uint i = 0; i < 4; i++) {
        x += 1;
    }
    return x;
}
```

Gas optimized function

```
function f(uint x) returns (uint) {
    x += 4;
    return x;
}
```

Improvements since DevCon: Highlights

- Overapproximation for `STATICCALL`, `BALANCE`, `EXTCODESIZE`, and `EXTCODEHASH`
- SMT solver now used to concretize symbolic expressions and continue execution
- Multiple solutions for symbolic expressions via iterative SMT solving
- Improved aliasing for symbolic and concrete addresses
- Constant propagation for symbolic values
- Branch depth and width can be limited
- Major CLI improvements for better UX

Improvements since DevCon: Highlights cont.

- Counterexamples are now validated via concrete execution
- Mapping of warnings and errors to source lines
- Better RPC caching&mocking framework via `--cache-dir`: persistent shared cache, single HTTPS session
- Allowing to restrict unknown address resolution to deployed contracts via `--only-deployed`
- Keccak computations extracted and asserted for correctness
- Better collection of storage reads for counterexamples
- Upcoming: queued symbolic execution for more robust exploration of large contracts

Improvements since DevCon: Minor Things

- Robust handling of invalid cheatcodes/precompiles
- Geth-compatible JSONL traces via `--json-trace`
- Updated forge and forge-std versions
- Simplified printed expressions in symbolic mode
- More robust calldata decoding, even with trailing bytes
- More aggressive calldata shrinking

Improvements since DevCon: Minor Things cont.

- New rewrite and simplification rules
- Read bytecode directly from files, supporting large contracts
- Better handling of symbolic `block.number` in library mode
- Optimized memory representation and operations
- Many bugfixes: infinite recursion, out-of-bounds reads, duplicate assertions, and more
- Test suite improvements and faster test running
- Documentation and tutorial updates

Overall Effect of Improvements

- Better first-time user experience
- Easier-to-understand counterexamples
- More robust handling of real-world contracts

Upcoming changes:

- Queued symbolic execution for better exploration of large contracts
- Better separation of symbolic/concrete execution modes
- Dumping execution traces' calldata for fuzzers to improve their corpus
  e.g. Echidna – but see Alex's talk later today!

Conclusions

- Fully-featured symbolic&concrete execution engine for EVM bytecode
- Easy-to-use: test setup via forge cheatcodes, standalone CLI
- Runs on any point in chain history via RPC
- Highly optimized IR with extensive simplifications
- Tight integration with an SMT solver for finding buggy inputs
- Used in practice by multiple tools and projects
- Library that can be integrated into other tools
- Actively maintained open-source project

Repository: `https://github.com/argotorg/hevm/`

User guide: `https://hevm.dev/`